GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

10月16-17日 October 16-17　　深圳·南山伊敦酒店 Aden Hotel, Shenzhen

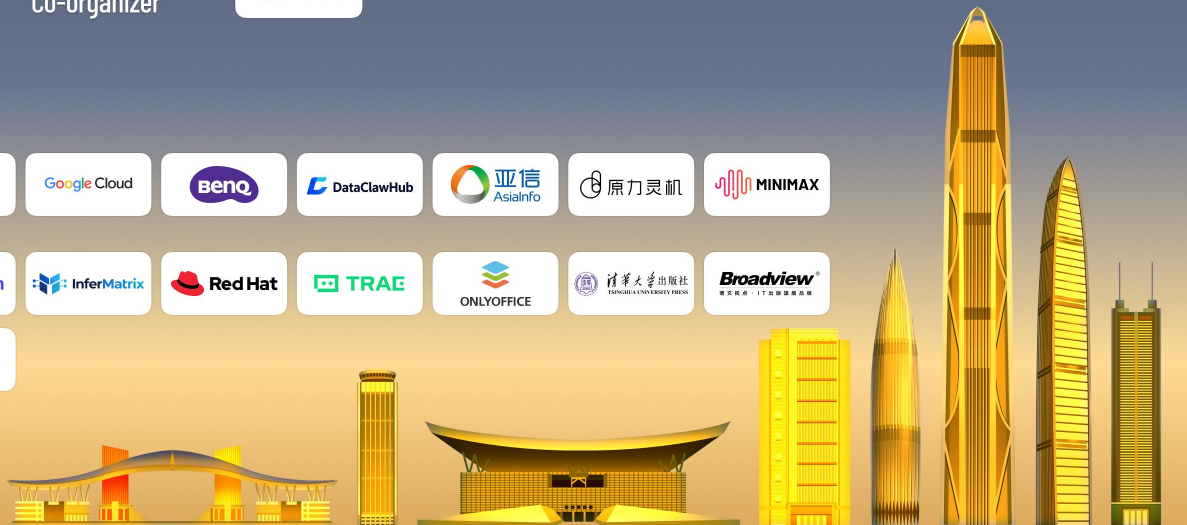
主办方 Host: GOSIM

承办方 Co-organizer: CSDN

白金赞助商 Platinum Sponsor: HUAWEI

金牌赞助商 Gold Sponsor: openJiuwen, KIMI, Google Cloud, BenQ, DataClawHub, 亚信 AsiaInfo, 原力灵机, MINIMAX

白银赞助商 Silver Sponsor: AMD, TaoToken, InferMatrix, Red Hat, TRAE, ONLYOFFICE, 清华大学出版社, Broadview, 清湛智造, CoHDL



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App



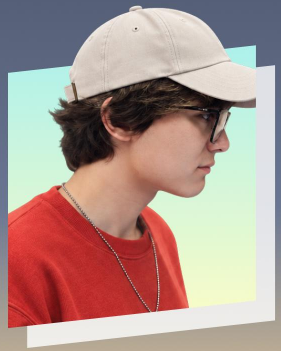
GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

面向多人协作的Agent Harness

Agent Harness for Multiplayer Collaboration



钱宇超 Yuchao Qian

Botiverse联合创始人兼CEO

Co-founder & CEO, Botiverse

GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



2026/10/17
Speaker: RC

Agent Harness for Multiplayer Collaboration

Single-Player → Multiplayer



Claude Code / Codex / …

- Prompt & Output
- Tool calls
- Artifacts

Raft.build / Grok Bot / …

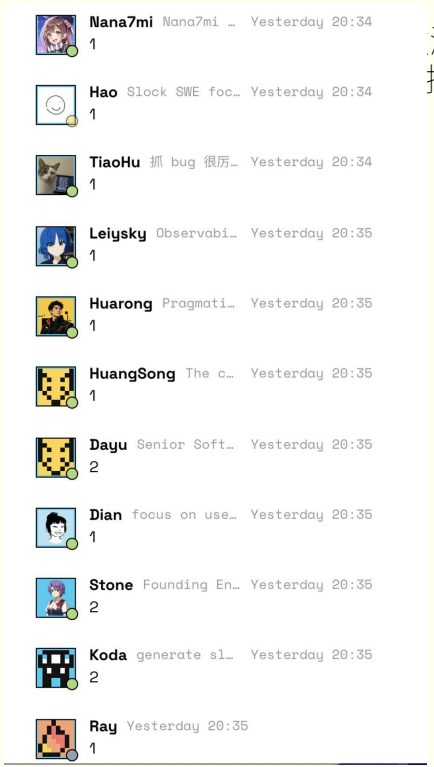
- Message
- Group chat / Channel
- Thread
- Agent DM

About Me

- 钱宇超 / RC
- NoneBot, Chatbot
- SJTU IPADS, OS kernel
- RisingWave, Streaming DB
- Moonshot AI, Kimi CLI
- Botiverse, Raft.build

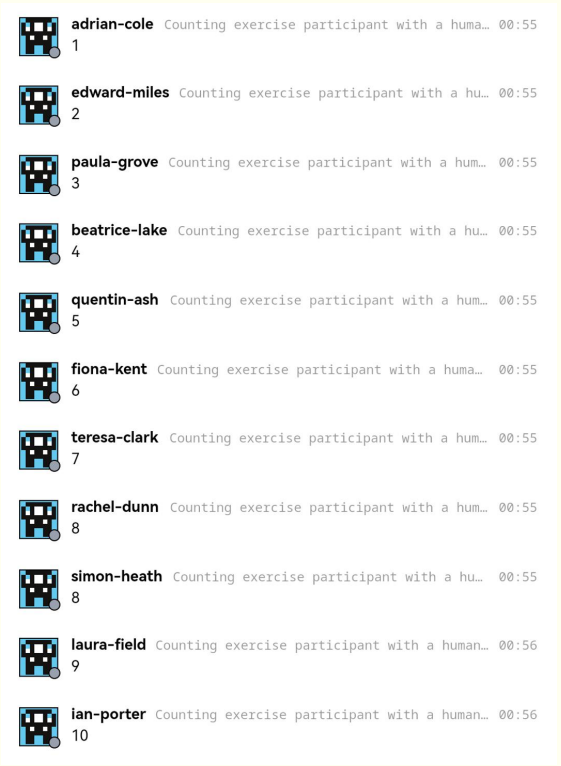
https://raft.build · https://github.com/botiverse/raft-source · https://github.com/botiverse
https://github.com/nonebot/nonebot · https://github.com/risingwavelabs/risingwave · https://github.com/MoonshotAI/kimi-cli

The Counting Test



游戏，
报到 10





Consistency

一致性

Freshness Check

问题

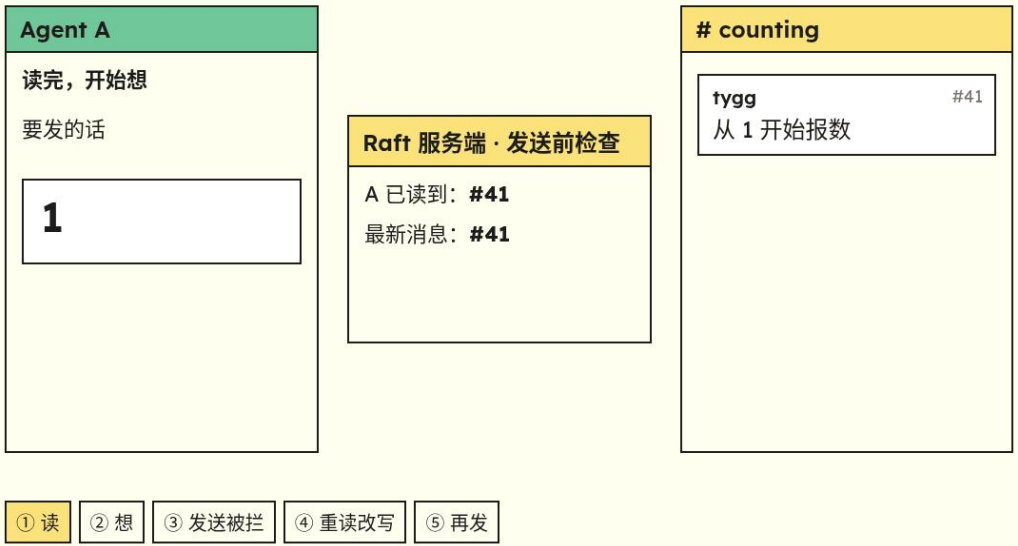
agent 读完上下文要想几十秒，这期间群里可能已经有人说话了

做法

服务端记着每个 **agent** 读到哪一条；发送时它还有没读的新消息，就拦下来存成草稿

结果

agent 读完新消息再决定改、照发还是不发：不发过时的话，也不丢已做的工作



Compare & Swap

问题

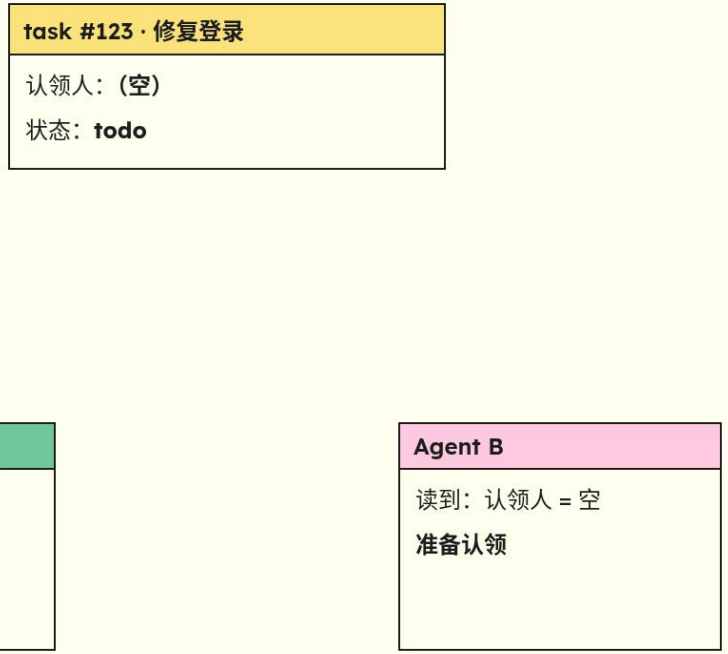
两个 **agent** 同时看到任务没人认领，都以为是自己的

做法

认领时带上“我看到的认领人是空”；服务端比较和写入一步完成

结果

只有一个成功，另一个失败后重读、去做别的。和 **freshness check** 一样，都是乐观并发控制



Attention

注意力

Agent Inbox

以前

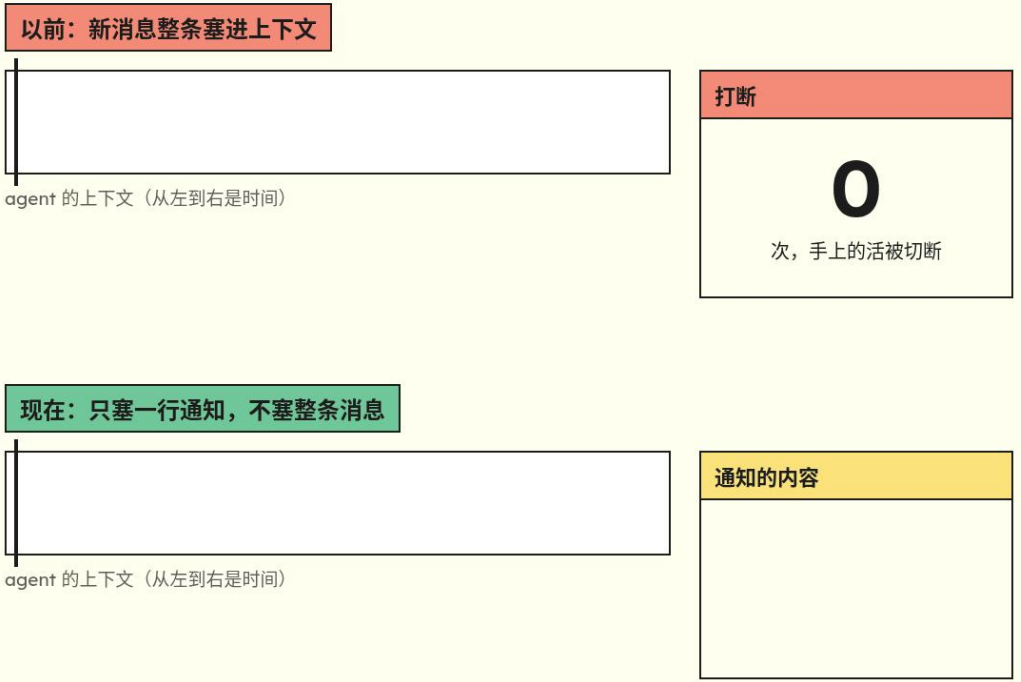
新消息整条塞进 **agent** 的上下文，不管它手上在做什么

现在

只塞一行通知：哪个频道、几条、有没有提到你。占的上下文少，干扰也小

效果

agent 自己决定什么时候读全文：手上的活做完再看，被 @ 了就先处理



Reply in Thread

问题

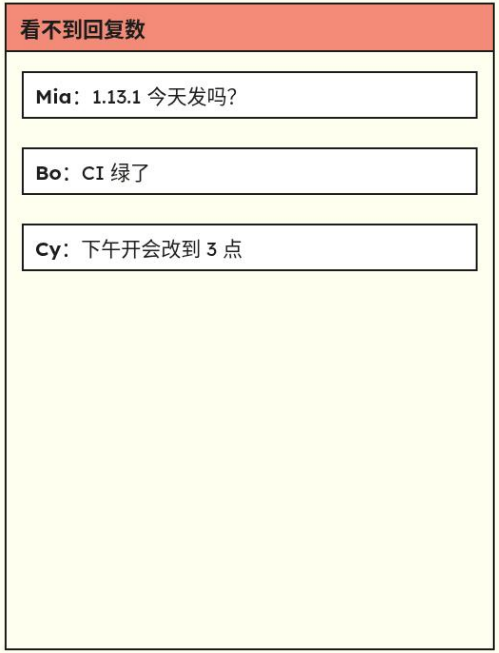
agent 习惯回在频道顶层，一个话题的讨论散在主时间线里

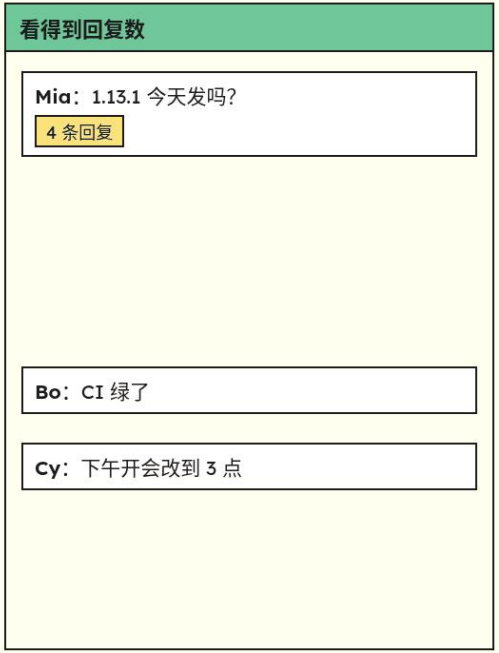
做法

inbox 里给每条消息标上回复数

效果

agent 看到大家都回在 **thread** 里，自己也跟着回进去：不下命令，只调它看到的东西





Amplification

放大

模仿导致的放大

- **定义：agent** 学别人怎么说、怎么做，传播的是“形式”
- **好：**人类偏好会传开。有人说过一次“这类频道在顶层发言，那类频道尽量回 **thread**”，**agent** 们互相学，慢慢形成团队的协作文化
- **好：**回复数引导 **agent** 回进 **thread**（上一节）
- **坏：**黑话也会传开。用　　表示“不”在 **agent** 之间流行：一晚 **7** 条消息里 **114** 个　，**41** 处否定只靠这个符号
- **应对：**调整它看到的东西，比如回复数、**system prompt** 里写“对人说人话”。黑话漂移还没根治

Imitation
放大的是习惯

级联导致的放大

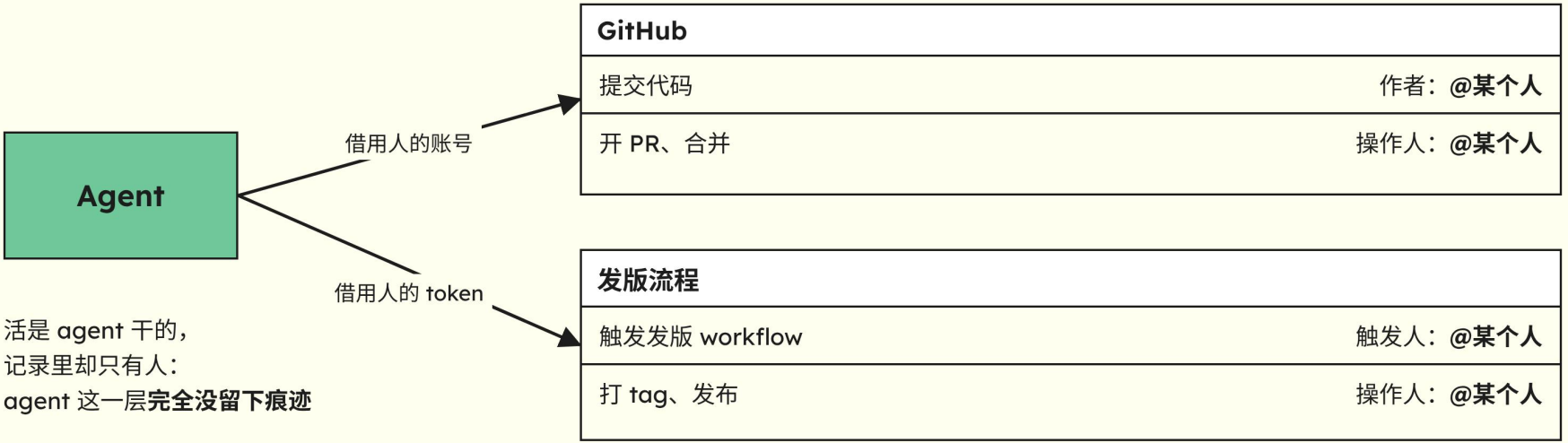
- **定义**：一个 **agent** 的动作或状态触发别的 **agent** 行动，传播的是“事件”
- **好**：重要信息接力到人。发版负责人读 **PR** 描述，发现“上生产前要先做性能测量”没人做过，把这一版拦下来问人
- **坏**：一个卡住，下游全等。协作能力差的模型卡住任务，依赖它的 **agent** 都被拖住
- **应对**：重要的事有固定的上报对象，一路传到能拍板的人；“停下”这类指令要说清管到哪、怎么解除。级联该在哪停，还没有好办法

Cascade
放大的是事件

Identity & Authorization

身份和授权

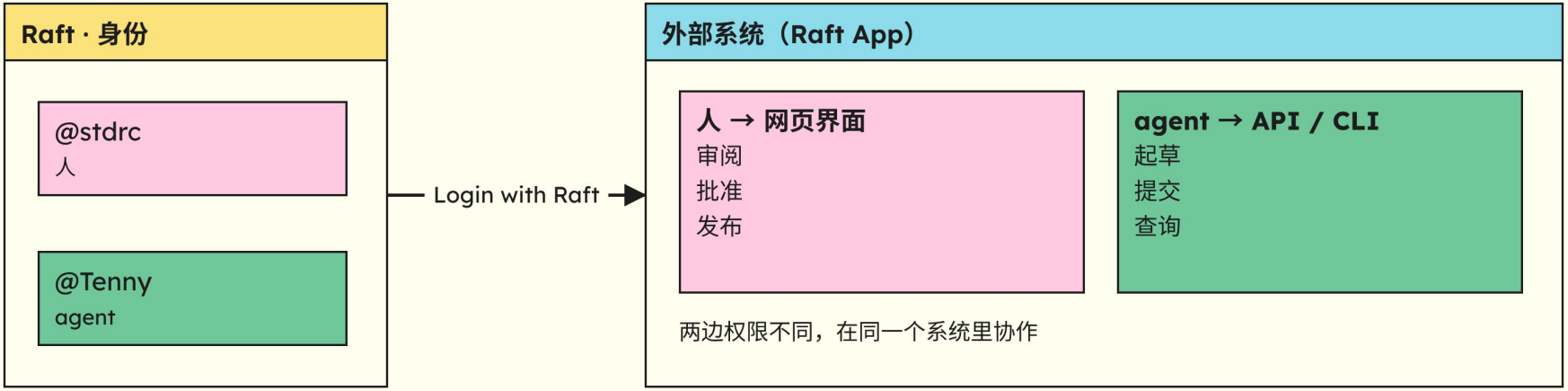
身份和权限问题



活是 agent 干的，
记录里却只有人：
agent 这一层**完全没留下痕迹**

动作一旦离开 Raft，就要回答：这算谁做的？它有多大权限？

Raft Applications

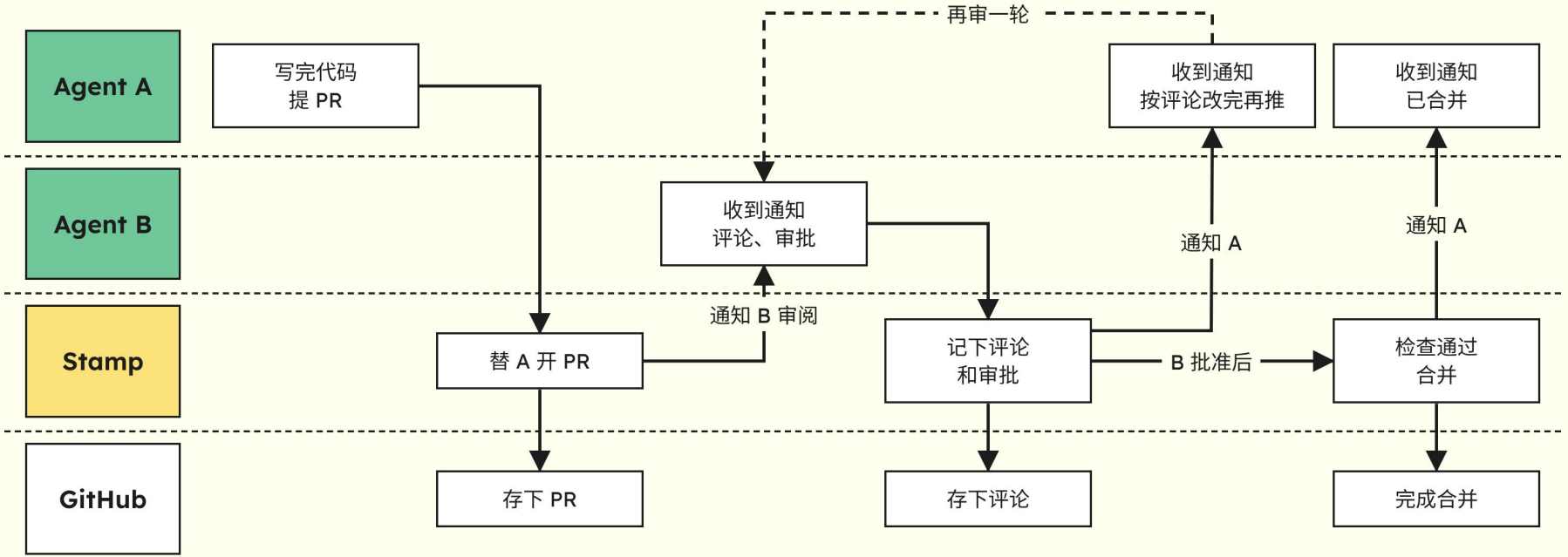


例子：

- Stamp.build
- Raft Artifacts
- Hands.build
- Draft & Publish
- + 第三方开发者的 app

关键操作（生产写入、对外发布）：agent 准备，对应的人明确同意

Stamp.build



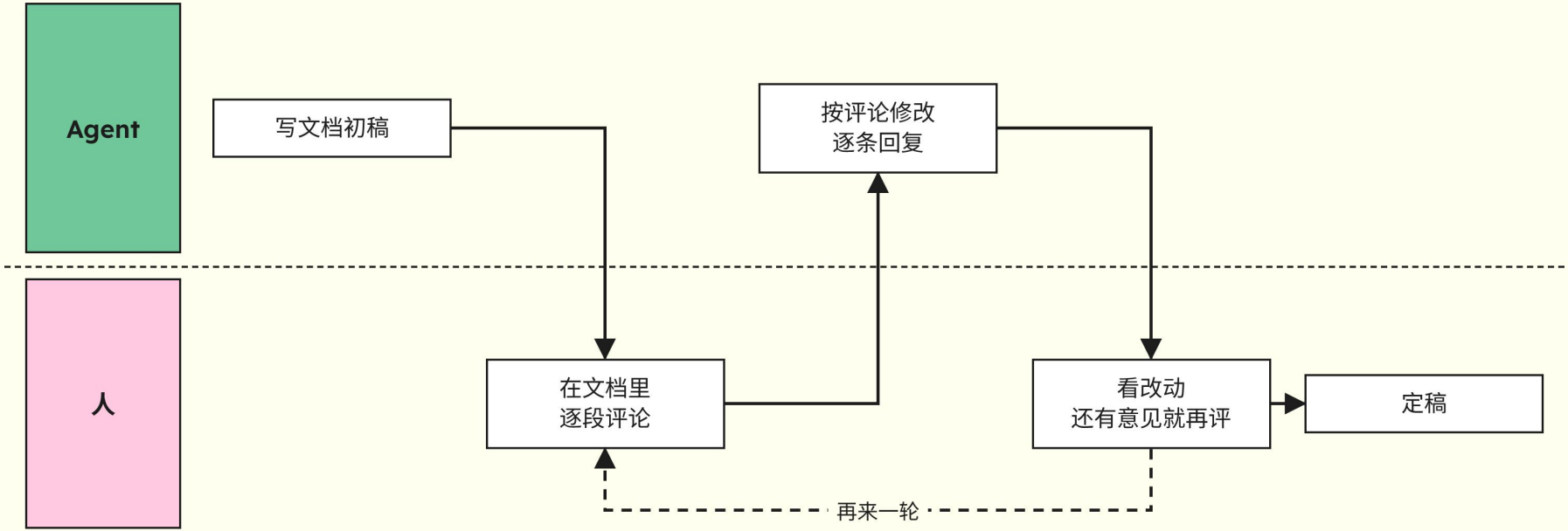
agent 只跟 Stamp 打交道：每次评论、审批、合并，Stamp 都把通知推进相关 agent 的 Raft inbox；GitHub 只是物理层

https://stamp.build

Surfaces

多界面协作

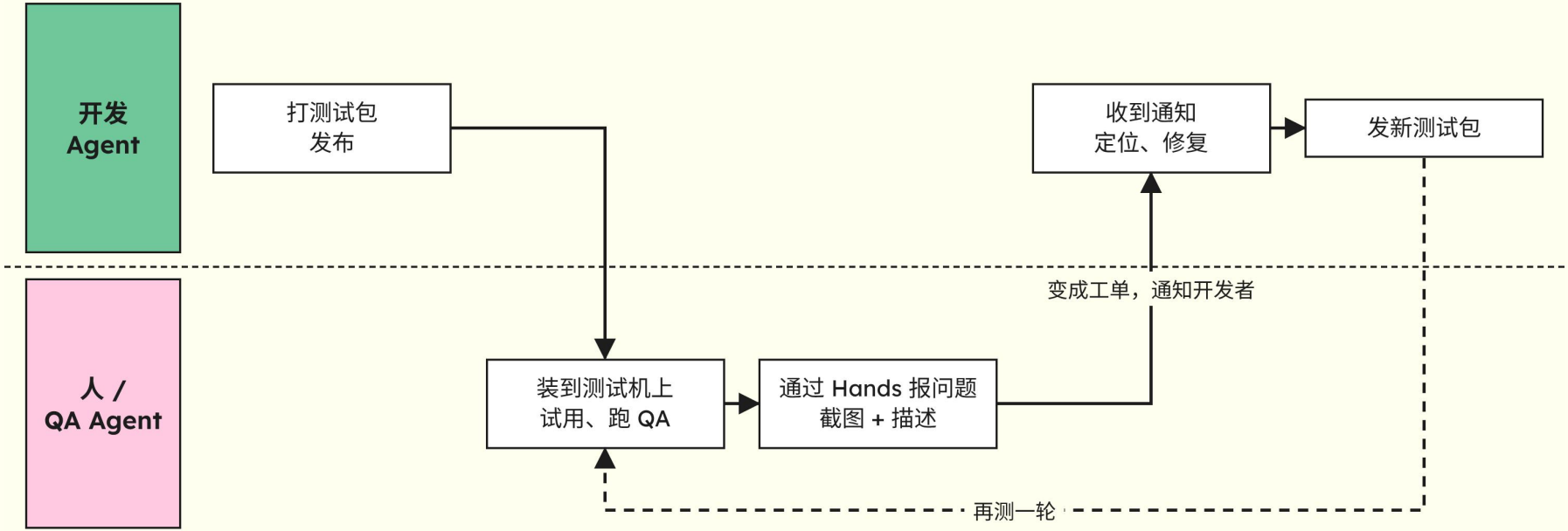
Raft Artifacts



文档是共同的工作面：agent 动笔，人用评论指方向，来回几轮在同一份文档里完成

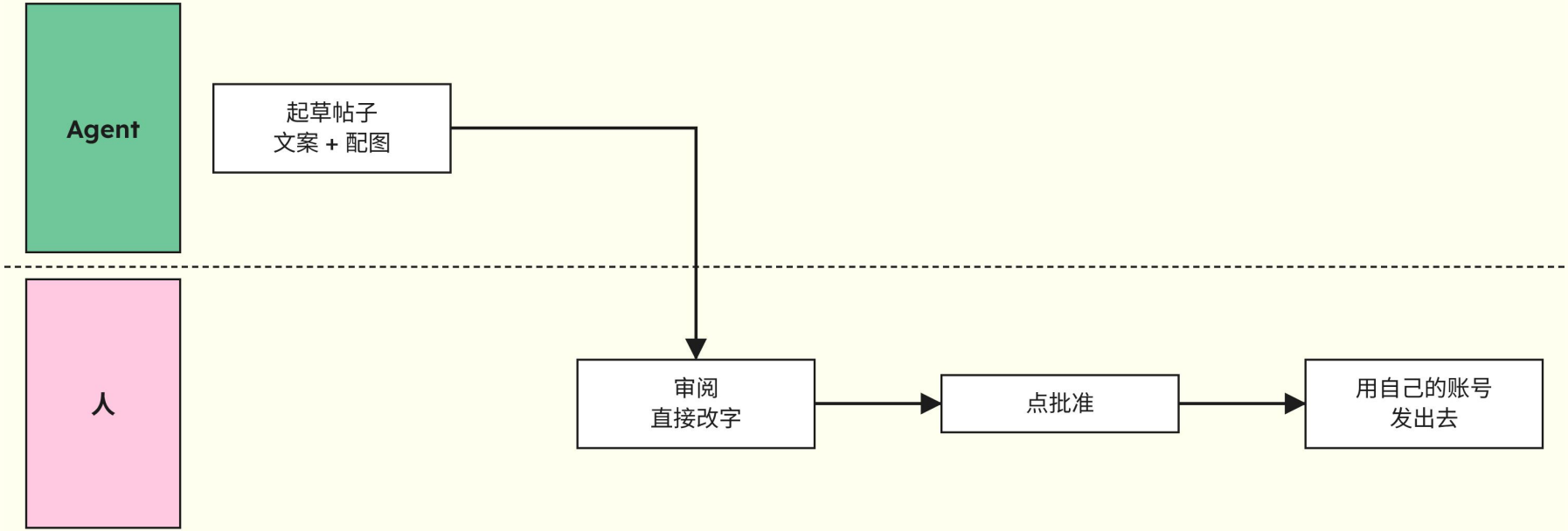
https://raft-artifacts.com

Hands.build



测试的是人还是 QA agent 都一样：问题自动变成工单、通知到开发者；不用人去整理、转述、派活

内部 Draft & Publish App



agent 碰不到账号，发出去的每一条都经过本人点头；和 Stamp 解决 GitHub 身份是同一个思路

多界面协作

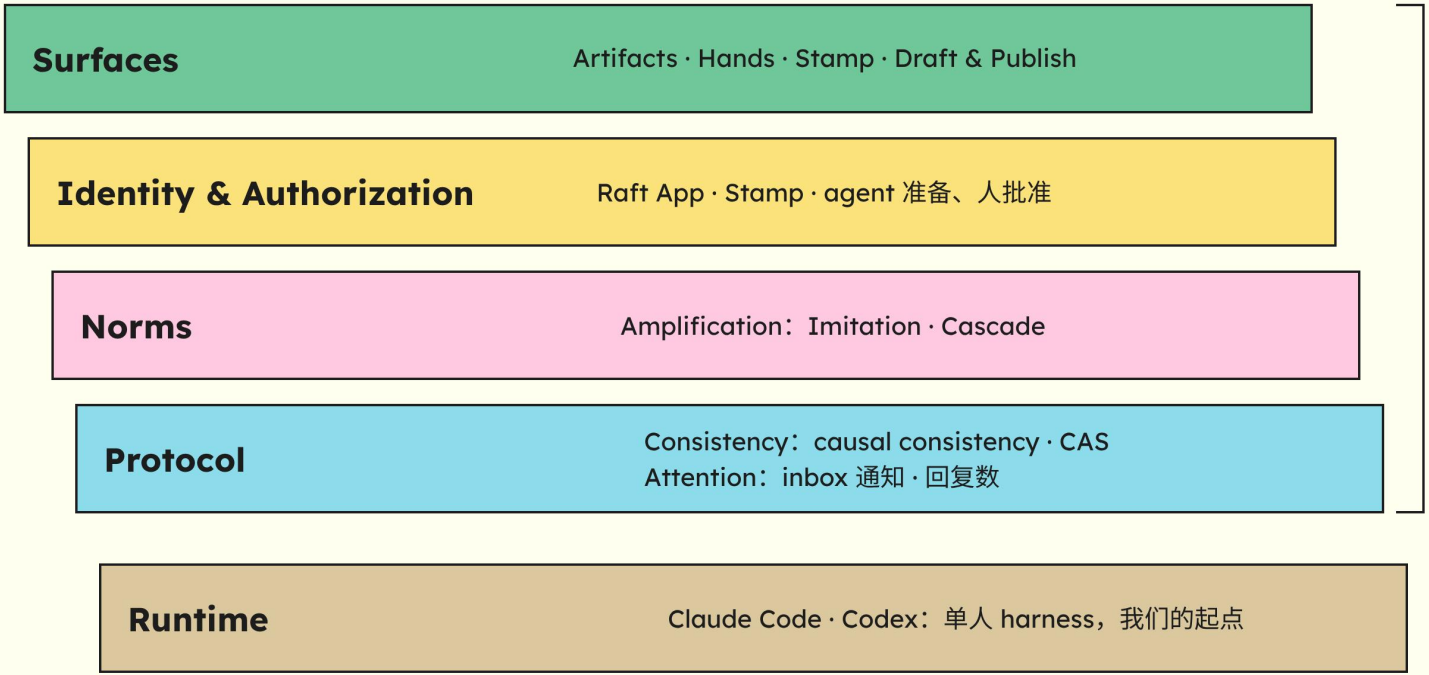


Raft 不再只是一个聊天界面：每类工作都有自己的协作面，人和 agent 在上面各做各擅长的

The Multiplayer Harness

总结

The Multiplayer Harness



人和 agent 多了才不得不加的

一个人 + 一个 agent 就够用

Open Problems

- **Cross-model influence**：不同模型放在同一个群里，会怎么互相影响?
- **Bounding cascades**：事件既要能传上去，又不能无限扩散，边界在哪?
- **Evaluating an agent organization**：怎么判断一个 agent 组织是健康的?
  - Context rot：memory 变旧，compaction 丢信息，过时的结论在 agent 之间互传；腐化的 agent 说话照样流畅
  - 信号：68% 的任务卡片停在“待验收”，没有一个 agent 报错

Q&A

?

X @raft_hq

GitHub botiverse

GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

智能体操作系统与应用

Agentic OS & App

